Аккредитованное образовательное частное учреждение
высшего образования
«Московский финансово-юридический университет МФЮА»
(МФЮА)

КАФЕДРА «[УКАЗАТЬ]»

КУРСОВАЯ РАБОТА

по дисциплине: «[УКАЗАТЬ]»

на тему: «Разработка модуля управления каталогом товаров для интернет-магазина»

Направление подготовки: [УКАЗАТЬ]
Профиль: [УКАЗАТЬ]

Автор работы: [ФИО]
Курс: [УКАЗАТЬ] Группа: [УКАЗАТЬ]
Руководитель: [ФИО, ДОЛЖНОСТЬ]
Подписи и даты: [ЗАПОЛНЯЮТСЯ УЧАСТНИКАМИ]

ТЕХНИЧЕСКИЙ ЧЕРНОВИК ДЛЯ ДОРАБОТКИ

[ГОРОД] — 2026

СОСТОЯНИЕ ЧЕРНОВИКА

Документ собран по фактически реализованному учебному проекту. Персональные реквизиты оставлены для заполнения. Служебные подписи, утверждение задания и оценка не подставлены.

Содержательная часть включает введение, три главы, заключение и технические первоисточники. Ссылки [Т1]–[Т6] временные: научная библиография, окончательная нумерация и её оформление ещё не завершены. Шесть технических документов не выполняют весь минимум источников МФЮА.

Базовые параметры: А4, поля слева 30 мм, справа 10 мм, сверху и снизу 20 мм, Times New Roman 14, интервал 1,5, абзац 1,25 см. После заполнения реквизитов нужно проверить итоговый объём 25–30 страниц без приложений и согласовать неоднозначности нумерации с преподавателем.

Иллюстрации получены из реального приложения. В рабочей базе по запросу пользователя 1200 синтетических товаров. Цены и остатки демонстрационные; исследование действующего предприятия и измерение экономического эффекта не проводились.

Проверка оригинальности в системе вуза не выполнялась. Перед сдачей студент должен проверить текст, источники и понимание кода. Эта служебная страница удаляется из итоговой версии.

СОДЕРЖАНИЕ

Введение.....4
1 Теоретические основы и анализ управления товарным каталогом.....5
1.1 Особенности ведения каталога товаров интернет-магазина.....5
1.2 Подходы к автоматизации и требования к модулю.....6
1.3 Обоснование выбора технологий.....7
2 Проектирование модуля управления каталогом.....9
2.1 Пользовательские сценарии и функциональная модель.....9
2.2 Проектирование базы данных и ограничений целостности.....10
2.3 Архитектура и пользовательский интерфейс.....12
3 Разработка и тестирование модуля.....14
3.1 Реализация операций с товарами и категориями.....14
3.2 Реализация поиска, фильтрации и веб-интерфейса.....15
3.3 Тестирование, результаты и ограничения.....15
Заключение.....18
Технические первоисточники (предварительный список).....19
Приложение А. Экраны и результаты демонстрации.....20

ВВЕДЕНИЕ

Каталог товаров объединяет сведения, необходимые для управления ассортиментом: названия, артикулы, принадлежность к категориям, цены и текущие остатки. В этой работе рассматривается локальный модуль для менеджера учебного интернет-магазина канцелярских товаров. Отдельное внимание уделено проверке вводимых данных и согласованности связей между товаром и категорией. Анализ распространённости проблем и общих тенденций электронной торговли должен быть дополнен изученной научной литературой; количественные сведения здесь не заявляются.

Цель работы — разработать и проверить модуль управления каталогом, позволяющий создавать, просматривать, изменять и удалять сведения о товарах и категориях, находить товары и отбирать их по заданным условиям.

Объект исследования — процесс ведения товарного каталога интернет-магазина. Предмет исследования — структуры данных, методы обработки запросов и программные средства автоматизации хранения, изменения и поиска сведений о товарах и категориях.

Для достижения цели определены задачи: сформировать требования к модулю; выбрать инструменты разработки; спроектировать структуру данных и пользовательские сценарии; реализовать серверные операции и веб-интерфейс; проверить положительные и отрицательные сценарии; описать результаты и ограничения.

Практический результат — приложение на Python с FastAPI, Jinja2, SQLAlchemy и SQLite. Оно предоставляет HTML-страницы и JSON API с общими операциями каталога. При работе использованы моделирование данных и сценариев, программирование и функциональное тестирование. Материалы записки организуются в три главы: теоретический анализ, проектирование, реализация и проверка.

1 ТЕОРЕТИЧЕСКИЕ ОСНОВЫ И АНАЛИЗ УПРАВЛЕНИЯ ТОВАРНЫМ КАТАЛОГОМ

1.1 ОСОБЕННОСТИ ВЕДЕНИЯ КАТАЛОГА ТОВАРОВ ИНТЕРНЕТ-МАГАЗИНА

В рамках учебной задачи товарный каталог рассматривается как набор записей об ассортименте. Отдельная запись описывает наименование, артикул, категорию, цену и текущее количество товара. Название помогает менеджеру найти и распознать товар, а артикул служит уникальным обозначением. Поэтому совпадение названий разных товаров допустимо, а совпадение артикулов запрещено. Например, две ручки одного типа с разными цветами могут иметь близкие названия, но разные артикулы.

Категория объединяет товары по назначению. Её идентификатор хранится у товара как ссылка на самостоятельную запись. Если менеджер переименовывает категорию, принадлежность товаров сохраняется: менять текст категории в каждой товарной записи не требуется. В рассматриваемом модуле у товара одна категория. Множественная классификация и иерархия вложенных категорий не включены в согласованные требования.

Цена и остаток имеют разные смыслы и проверяются отдельно. Нулевая цена допускается, а отрицательная цена не принимается. Нулевой остаток означает отсутствие товара; отрицательный или дробный остаток не соответствует принятой модели поштучного учёта. Фильтр наличия зависит только от количества. Установка количества в карточке фиксирует текущее состояние, но не объясняет причину его изменения. Для приходов, списаний и резервирования нужна отдельная модель операций, которой в первой версии нет.

Основная роль пользователя — менеджер каталога. Для него важны создание и изменение записей, поиск нужной позиции, обзор цены и наличия, а также понятная реакция на неправильный ввод. В отличие от покупательского

интерфейса модуль ориентирован на редактирование ассортимента. Заказы, корзина, платежи и доставка находятся за его границами. Учебный ассортимент не описывает деятельность обследованного предприятия: данные созданы специально для демонстрации.

1.2 ПОДХОДЫ К АВТОМАТИЗАЦИИ И ТРЕБОВАНИЯ К МОДУЛЮ

Для выбора формы решения рассмотрены три организационных подхода: ведение перечня в таблице, применение готовой системы магазина и разработка отдельного модуля. Сравнение носит проектный характер и опирается на требования этой работы; испытание конкретных коммерческих продуктов не проводилось.

Перечень в таблице подходит для первоначального описания структуры данных. В нём можно представить строки товаров и столбцы свойств, однако правила уникальности и связанные категории нужно организовывать отдельно. Для учебной задачи таблица служит удобным исходным представлением, но не заменяет требуемую реализацию HTTP-запросов, серверной проверки и веб-форм.

Готовая система магазина представляет другой путь: требуемые функции необходимо найти, настроить или расширить в существующем продукте. Этот вариант может быть рассмотрен в прикладной задаче выбора системы, но в данной работе требуется самостоятельно спроектировать и объяснить модуль на Python и FastAPI. Поэтому заимствование готового приложения не является выбранным способом выполнения проекта. Количественная оценка стоимости и времени внедрения не проводилась.

Отдельный веб-модуль позволяет явно связать каждое требование со структурой данных, обработчиком и проверкой. Цена такого решения — необходимость самостоятельно реализовать интерфейс, обработку ошибок и ограничения. В учебном проекте это соответствует цели: получить обозримый исходный код и воспроизводимую демонстрацию. Сравнение не доказывает

универсального преимущества собственной разработки перед готовыми системами.

Таблица 1 — Проектное сравнение подходов

Подход	**Что организуется**	**Соответствие задаче**
Таблица товаров	Столбцы свойств, правила ввода и связи нужно настроить	Подходит для исходного перечня; HTTP-модуль не реализует
Готовая система	Выбор продукта, настройка или расширение	Возможный прикладной путь; самостоятельная разработка не показана
Отдельный модуль	Собственные схема, операции, формы и проверки	Выбран для изучения Python и FastAPI

К функциональным требованиям отнесены CRUD товаров и категорий, карточка товара, поиск по названию, совместные фильтры категории, цены и наличия, сортировка и пагинация. Требования к корректности включают уникальность артикула и названия категории, проверку денежных значений и количества, запрет удаления занятой категории и сохранность данных после повторного запуска. Понятные ошибки, сохранение ввода и работа форм без JavaScript рассматриваются как требования к пользовательским сценариям. Все эти требования связаны с фактическими проверками в протоколе проекта.

1.3 ОБОСНОВАНИЕ ВЫБОРА ТЕХНОЛОГИЙ

Для проекта выбран Python 3.12. FastAPI принимает HTTP-запросы и предоставляет документацию JSON API. HTML формируется на сервере с Jinja2: обработчик передаёт объект запроса, имя шаблона и словарь данных в TemplateResponse [T1]. Это позволяет использовать обычные формы без отдельного приложения в браузере.

Для доступа к данным применён SQLAlchemy. Сессия связывает объекты и операции с базой; завершение транзакции выполняется явно [T2]. SQLite выбрана проектным решением для локальной демонстрации: отдельный сервер базы не требуется. Это решение не подтверждает пригодность для произвольной нагрузки. Зависимости закреплены в requirements-файлах и проверены в Python 3.12.

Pydantic используется для входных схем. Строгая проверка отдельных полей позволяет отвергать данные неподходящего типа вместо их неявного преобразования [T5]. В проекте это относится к JSON-полям количества и идентификатора категории. HTML-форма отправляет строки; адаптер формы переводит допустимую целочисленную запись в число до проверки той же схемой.

2 ПРОЕКТИРОВАНИЕ МОДУЛЯ УПРАВЛЕНИЯ КАТАЛОГОМ

2.1 ПОЛЬЗОВАТЕЛЬСКИЕ СЦЕНАРИИ И ФУНКЦИОНАЛЬНАЯ МОДЕЛЬ

Основной пользователь — менеджер каталога. Он создаёт категорию, добавляет относящийся к ней товар, проверяет карточку, находит товар по названию, меняет цену и остаток. При удалении открывается подтверждение. Если категория содержит товары, интерфейс объясняет запрет и предлагает открыть их список.

Поиск сочетается с выбором категории, нижней и верхней границами цены, признаком наличия и сортировкой. Наличие означает строго положительный остаток. Границы цены включаются в результат. Список разделён на страницы по десять записей. Параметры отбора сохраняются при переходе по страницам. Сценарии и их соответствие требованиям раскрыты в DESIGN.md.

Таблица 2 — Связь требований с проверками

Требование	**Реализация**	**Признак корректности**
CRUD товара	Форма, карточка, API	Изменения видны после чтения; ID сохраняется
Категории	Справочник и связь	Повторное имя и удаление занятой категории отклоняются
Поиск	Нормализованное имя	Русский регистр не меняет результат
Фильтры	Условия общего запроса	Категория, цена и наличие применяются вместе
Пагинация	LIMIT 10 и OFFSET	120 страниц для 1200 товаров
Неправильный ввод	Схемы и форма	Статус 422; заполненные значения сохранены
Данные после старта	Файловая SQLite	Повторное открытие не удаляет записи

2.2 ПРОЕКТИРОВАНИЕ БАЗЫ ДАННЫХ И ОГРАНИЧЕНИЙ ЦЕЛОСТНОСТИ

Выделены сущности «Категория» и «Товар» со связью один ко многим. Категория имеет идентификатор, название и необязательное описание. Товар содержит идентификатор, название, артикул, описание, идентификатор категории, цену, остаток, время создания и изменения. Товар обязательно связан с существующей категорией; удаление занятой категории запрещено внешним ключом RESTRICT и операцией каталога.

Цена хранится целым числом копеек. Входная строка рублей разбирается через Decimal: принимаются запятая или точка и до двух дробных знаков. Значения с большей точностью не округляются автоматически, а отклоняются. При выводе цена снова представляется строкой рублей с двумя знаками. Так точность денежного значения не зависит от двоичного float.

Остаток — неотрицательное целое в диапазоне SQLite. На уровне входных схем и моделей отклоняются дробные, логические и недопустимо большие значения. CHECK-ограничения SQLite дополнительно защищают обязательные свойства при прямой записи в базу. Уникальность артикула и нормализованного названия категории обеспечивается базой и предварительными проверками.

Для поиска используется дополнительное нормализованное название: NFKC, удаление крайних пробелов и casefold. В документации Python NFKC определена как совместимостное разложение с последующей канонической композицией [T4]. Проектная последовательность преобразований и поиск русских названий проверены тестами. Оригинальное название сохраняется для отображения. Полный словарь полей и ER-схема находятся в DESIGN.md.

Таблица 3 — Основные поля модели данных

Сущность и поле	**Представление**	**Правило**
Category.id	Целое	Первичный ключ
Category.name	Строка до 100 символов	Непустое; уникальна нормализованная форма
Category.description	Текст или NULL	Не более 2000 символов
Product.id	Целое	Первичный ключ
Product.name	Строка до 200 символов	Непустое; нормализованная копия для поиска
Product.sku	Строка до 64 символов	Уникальный; латинские буквы, цифры, дефис, подчёркивание
Product.category_id	Целое	Внешний ключ; категория существует; RESTRICT
Product.price_kopecks	Целые копейки	От 0 до 2^63 − 1; при вводе рубли строкой
Product.quantity	Целое количество	От 0 до 2^63 − 1
Product.description	Текст или NULL	Не более 2000 символов
Product.created_at, updated_at	Дата и время	Хранятся в UTC; обновление при изменении

Связь таблиц: categories.id → products.category_id, один ко многим. Удаление занятой категории запрещено. Нормализованные поля служебные и не входят в публичную схему API.

2.3 АРХИТЕКТУРА И ПОЛЬЗОВАТЕЛЬСКИЙ ИНТЕРФЕЙС

Компоненты организованы по назначению: database.py создаёт движок и сессии; models.py описывает таблицы и ограничения; schemas.py проверяет входные данные; catalog.py реализует операции; routers/api.py и routers/pages.py

адаптируют эти операции к JSON и HTML. Шаблоны отвечают за представление, локальные CSS — за оформление. Небольшой JavaScript переводит UTC-даты в местное время браузера.

Таблица 4 — Ответственность компонентов

Компонент	**Назначение**
database.py	Движок, внешний ключ, сессия на запрос
models.py / schemas.py	Таблицы, ограничения и проверка входных данных
catalog.py	Общие операции, транзакции, поиск и фильтры
routers/api.py	JSON-контракт и статусы HTTP
routers/pages.py	HTML-формы, сообщения и перенаправления
templates / static	Представление, оформление и местное время
demo_data.py / seed_demo.py	Воспроизводимые примеры, backup и загрузка

Общие операции предотвращают расхождение правил между интерфейсом и API. Например, запрет удаления занятой категории действует в обоих вариантах обращения. Каждый запрос получает собственную сессию. Запись завершается commit; при известных конфликтах целостности выполняется rollback [T2]. Схема компонентов подготовлена в DESIGN.md.

На широком экране меню расположено слева, фильтры — над таблицей. На узком экране меню переходит наверх, поля располагаются последовательно. Широкая таблица прокручивается внутри своего контейнера. Формы сопровождаются видимыми подписями и подсказками, сообщения об ошибках сохраняют введённый текст. Удаление отделено от обычного просмотра.

3 РАЗРАБОТКА И ТЕСТИРОВАНИЕ МОДУЛЯ

3.1 РЕАЛИЗАЦИЯ ОПЕРАЦИЙ С ТОВАРАМИ И КАТЕГОРИЯМИ

Входной точкой служит app/main.py. При запуске lifespan создаёт отсутствующие таблицы. Путь к файлу базы вычисляется от папки проекта, поэтому не зависит от текущей папки терминала. Повторная инициализация не удаляет записи; автоматического изменения существующей схемы не предусмотрено.

JSON API содержит десять операций: получение списка, создание, получение одной записи, изменение и удаление для товаров и категорий. При создании возвращается статус 201, при успешном удалении — 204 без тела. Отсутствующая запись даёт 404, конфликт уникальности или попытка удалить занятую категорию — 409, неправильный ввод — 422. PUT требует все обязательные редактируемые поля. Примеры запросов и ответов сохранены в STAGE_5.md.

Создание товара выполняется после проверки его полей, существования категории и уникальности артикула. При редактировании эти проверки выполняются до изменения сохранённого объекта. Регистр артикула приводится к верхнему. Если при записи всё же возникает конфликт ограничения базы, транзакция откатывается; другие записи не должны изменяться.

На стороне страниц успешный POST завершается ответом 303 и адресом результата. Затем браузер получает карточку или список отдельным GET. Такая схема согласуется с назначением 303 See Other для перехода после POST к отдельно доступному ресурсу [T6]. Открытие страницы подтверждения через GET не меняет данные.

3.2 РЕАЛИЗАЦИЯ ПОИСКА, ФИЛЬТРАЦИИ И ВЕБ-ИНТЕРФЕЙСА

Отбор товаров реализован в общей функции list_products. К запросу добавляются только выбранные условия. Строка поиска сравнивается с нормализованным названием; символы % и _ экранируются как буквальный ввод, а не шаблон SQL LIKE. Условия категории, цены и остатка применяются совместно. Порядок сортировки выбирается из фиксированного перечня, а идентификатор используется для однозначного порядка при равных значениях.

Количество найденных записей вычисляется отдельным запросом. Для страницы применяются limit и offset с размером десять записей. Форма поиска использует GET, поэтому её состояние видно в адресе. При ошибочном диапазоне цены возвращается HTML с кодом 422 и заполненными полями. Отсутствие результатов показано отдельным состоянием.

Jinja2 использует общий базовый шаблон с навигацией и отдельные шаблоны списка, карточки, форм и подтверждений. При включённом автоматическом экранировании Jinja обрабатывает переменные, кроме специально помеченных безопасными значений [T3]. В проекте пользовательский текст не получает такую отметку. Тесты проверяют, что введённые теги выводятся как текст. Основные сценарии форм работают при отключённом JavaScript.

3.3 ТЕСТИРОВАНИЕ, РЕЗУЛЬТАТЫ И ОГРАНИЧЕНИЯ

Проверка проведена 21.10.2026 в облачной Linux-среде. Автоматические проверки используют отдельный файл SQLite для каждого теста. Всего пройдено 154 теста: 35 проверок данных, 81 проверка API, 31 проверка HTML и 7 проверок загрузчика демонстрационного набора. Сохранено одно предупреждение о поддерживаемом устаревающем режиме HTTPX в TestClient. Оно не скрывалось и не привело к отказу проверок.

Дополнительно выполнены 14 сценариев в Chromium 154.0.8037.57 с экранами 1440 × 1000 и 390 × 844. Проверены создание и изменение через

формы, валидация цены с сохранением полей, совместные фильтры, пагинация, защита занятой категории, отмена и подтверждение удаления, отсутствие ошибок JavaScript и работа формы без JavaScript. Сохранены девять снимков реальных страниц на учебном ассортименте. Полный перечень — в browser-report.json и STAGE_7.md.

Браузерная проверка выявила расширение мобильной страницы скрытой подписью таблицы. Позиционирование подписи ограничено прокручиваемым контейнером; после исправления ширина документа не превышала ширину экрана в проверенных сценариях списка и формы. Также исправлено выравнивание последнего столбца. Дополнительные тесты проверяют недопустимо большие идентификаторы HTML-адресов.

Сохранность данных подтверждена повторным открытием базы другим процессом и ранее выполненным HTTP-прогоном с перезапуском на этапе 5. На этапах 5–7 рабочая база не использовалась для тестовых записей. Позднее по прямому запросу пользователя в неё добавлены 1200 демонстрационных товаров в 12 категориях после резервного копирования. Повторный запуск не добавил дублей; ограничения внешних ключей и целостность SQLite проверены. Пользователь подтвердил работу обновлённого интерфейса на своём Mac; браузер и его версия не были указаны.

Для проверки большого списка подготовлен воспроизводимый набор из 1200 товаров: по 100 в каждой из 12 категорий, 133 товара с нулевым остатком. Цены находятся в диапазоне 19,00–346,10 рубля. Они условные и не взяты из прайс-листа. Загрузчик использует стабильные артикулы, повторно применяет ограничения схем, создаёт SQLite backup перед записью и добавляет недостающее одной транзакцией. Пользовательские записи и правки существующих артикулов сохраняются. Семь специальных тестов проверяют также откат при сбое и работу резервного копирования с WAL.

На этом наборе в Chromium выполнено ещё шесть сценариев чтения: числа записей API, переход по страницам, последняя страница, совместные фильтры, счётчики категорий и ширина мобильного списка. Отдельный отчёт

находится в figures/large-catalog/report.json. Замеров пропускной способности и доказательства промышленной нагрузки эти сценарии не дают.

Таблица 5 — Фактические результаты проверки

Набор	**Количество**	**Результат**
База данных	35 тестов	Пройдены
JSON API	81 тест	Пройдены
HTML-страницы	31 тест	Пройдены
Загрузчик примеров	7 тестов	Пройдены
Браузер, основные сценарии	14 сценариев	Пройдены в Chromium
Браузер, 1200 товаров	6 сценариев	Пройдены в Chromium
SQLite большого набора	integrity_check, foreign_key_check	ok; ошибок связей нет

Полученные результаты подтверждают выполнение согласованных функциональных сценариев. Измерение производительности и нагрузочное тестирование не проводились; оценка ускорения работы менеджера не заявляется. В первой версии отсутствуют авторизация, журнал движений запасов и миграции схемы. Направления развития — роли пользователей, учёт операций с остатками, управляемые миграции и проверка работы при одновременном обращении нескольких менеджеров.

ЗАКЛЮЧЕНИЕ

В ходе проектирования и разработки создан локальный модуль управления товарным каталогом. Реализованы операции с товарами и категориями, просмотр подробной карточки, поиск, комбинированные фильтры, сортировка и пагинация. Разработаны JSON API и веб-интерфейс, использующие общие операции и ограничения.

Целостность данных поддерживается проверкой входных схем, правилами моделей и ограничениями SQLite. Цена хранится в копейках, остаток — целым неотрицательным числом. Категория с товарами защищена от удаления. Проверки выявили и позволили исправить дефекты мобильного отображения; итоговые 154 автоматических теста прошли. Проверены 14 сценариев интерфейса и ещё 6 сценариев большого каталога.

Приложение пригодно для демонстрации согласованных учебных сценариев. Доказательство работы при промышленной нагрузке, внедрение в организацию и экономический эффект в рамках проекта не получены. Для завершения курсовой необходимо дополнить теоретический анализ, собрать и изучить научные источники, оформить полный документ по шаблонам МФЮА и выполнить установленную вузом проверку.

ТЕХНИЧЕСКИЕ ПЕРВОИСТОЧНИКИ (ПРЕДВАРИТЕЛЬНЫЙ СПИСОК)

[T1] FastAPI. Templates. Исходный документ в официальном репозитории. URL: https://raw.githubusercontent.com/fastapi/fastapi/master/docs/en/docs/advanced/templates.md (дата обращения: 21.10.2026).

[T2] SQLAlchemy. Session Basics. Исходный документ в официальном репозитории. URL: https://raw.githubusercontent.com/sqlalchemy/sqlalchemy/main/doc/build/orm/session_basics.rst (дата обращения: 21.10.2026).

[T3] Jinja. Template Designer Documentation. Исходный документ в официальном репозитории. URL: https://raw.githubusercontent.com/pallets/jinja/main/docs/templates.rst (дата обращения: 21.10.2026).

[T4] Python 3.12. unicodedata — Unicode Database. Исходный документ в официальном репозитории. URL: https://raw.githubusercontent.com/python/cpython/3.12/Doc/library/unicodedata.rst (дата обращения: 21.10.2026).

[T5] Pydantic. Strict Mode. Исходный документ в официальном репозитории. URL: https://raw.githubusercontent.com/pydantic/pydantic/main/docs/concepts/strict_mode.md (дата обращения: 21.10.2026).

[T6] HTTP Working Group. RFC 9110: HTTP Semantics. Исходный документ в официальном репозитории. URL: https://raw.githubusercontent.com/httpwg/http-core/main/rfc9110.xml (дата обращения: 21.10.2026).

Список требует дополнения изученной научной литературой и окончательного оформления по ГОСТ. Реестр проверки источников: docs/coursework/SOURCES.md.

ПРИЛОЖЕНИЕ А. ЭКРАНЫ И РЕЗУЛЬТАТЫ ДЕМОНСТРАЦИИ

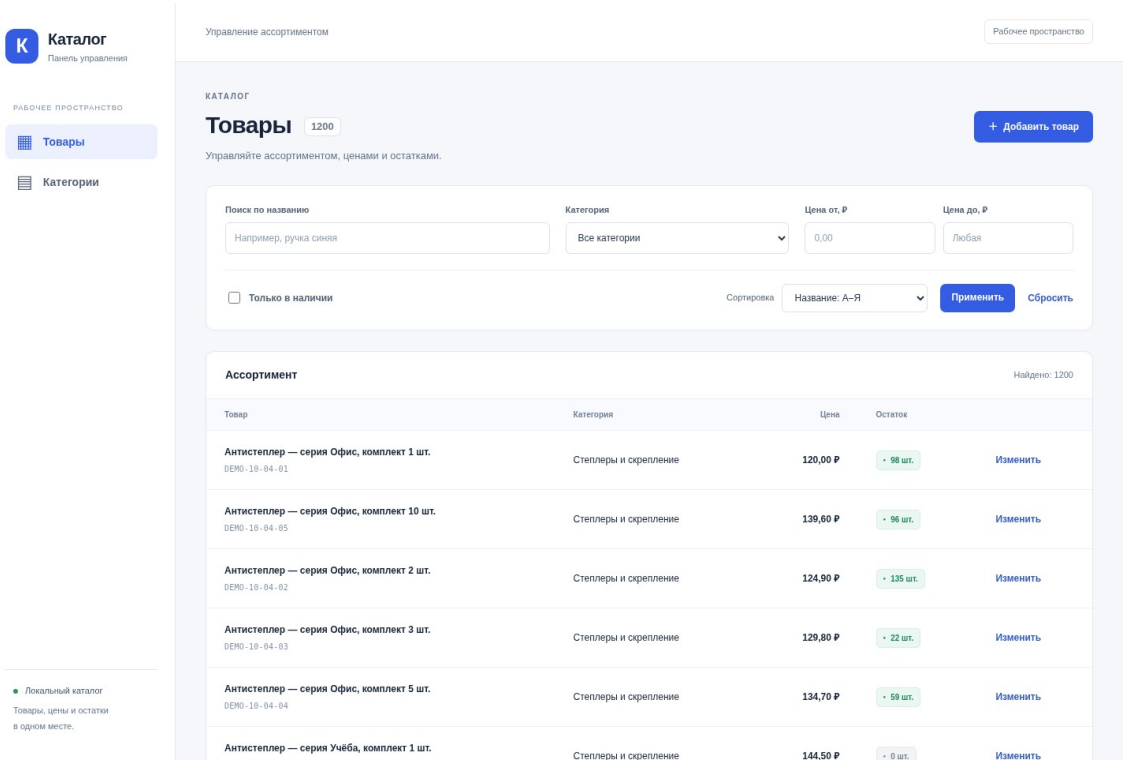


Рисунок А.1 — Каталог из 1200 демонстрационных товаров

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

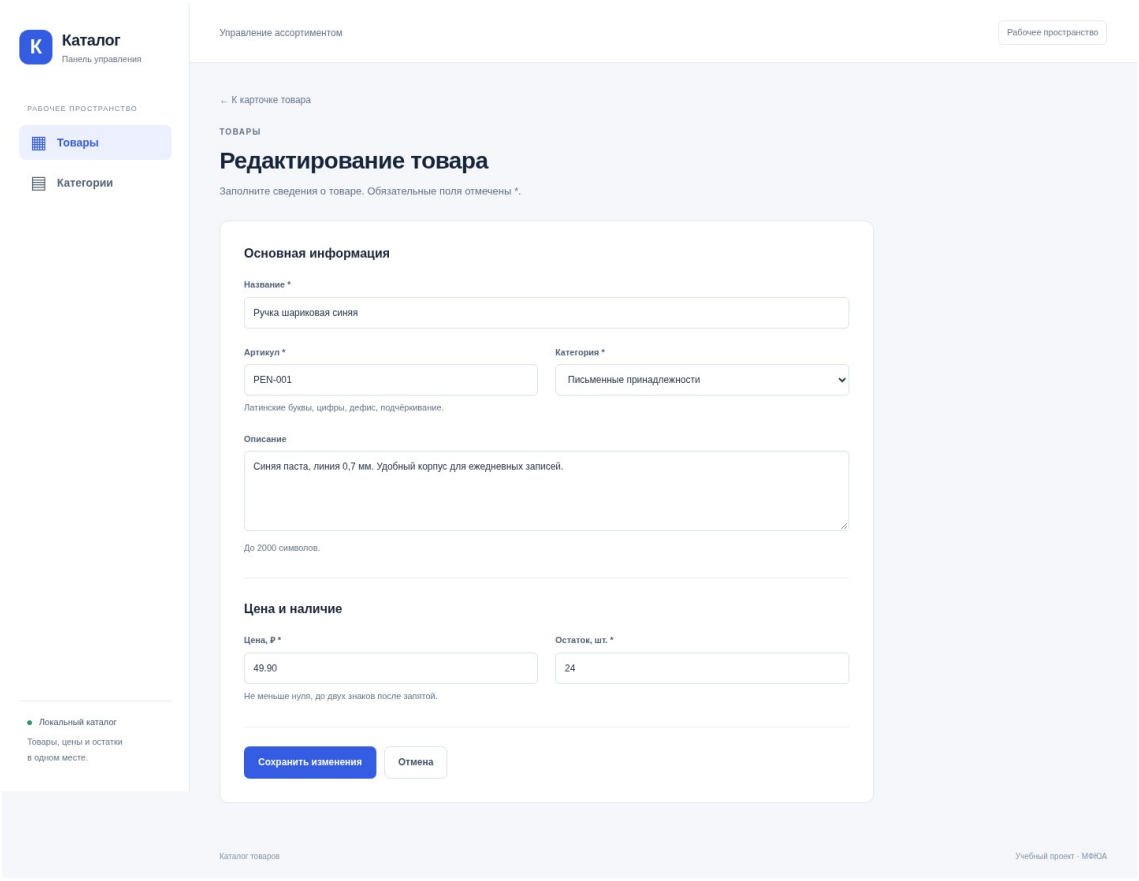


Рисунок А.2 — Форма редактирования товара

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

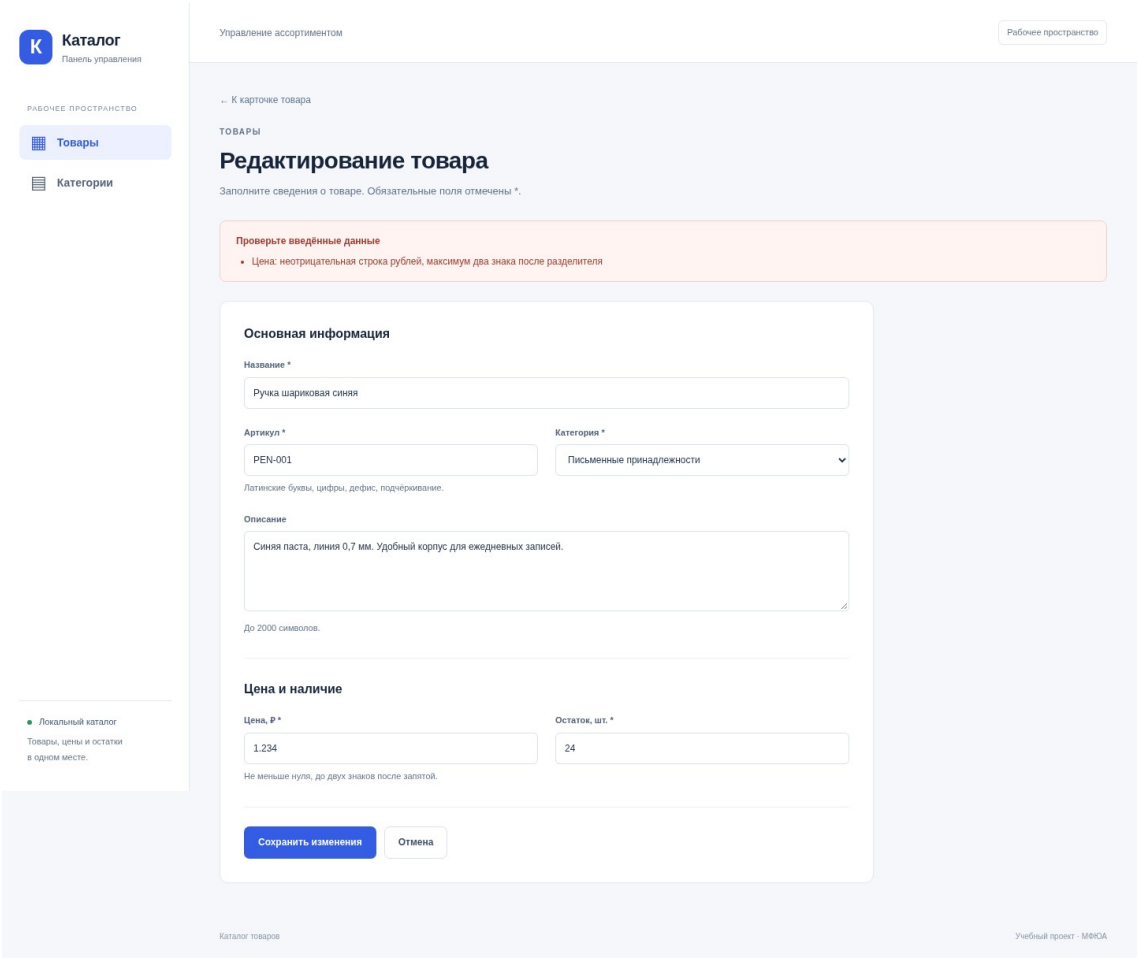


Рисунок А.3 — Сохранение ввода при неправильной цене

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.

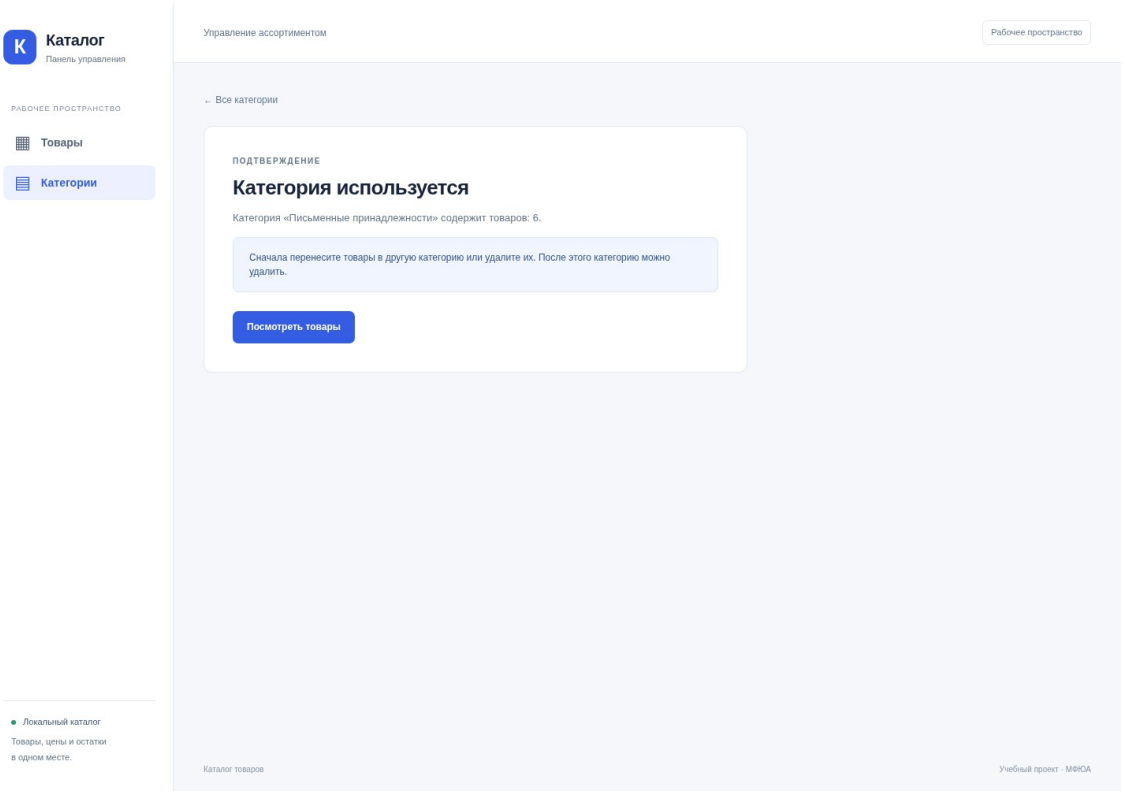


Рисунок А.4 — Запрет удаления категории с товарами

Источник: собственная разработка; реальная браузерная проверка на демонстрационных данных.